MyBiz2Flow — Application Documentation

MyBiz2Flow is a multi-tenant CRM & Invoicing SaaS built for UAE small businesses. It covers the full sales lifecycle — leads → quotations → invoices → payments — plus expense tracking, VAT-aware financial reports, and a platform-level "SaaS admin" layer for managing tenant companies and subscriptions.

Stack

| Layer | Technology | |---|---| | Backend | Laravel 13 (PHP 8.3), MySQL | | Auth | Laravel Sanctum (two separate guards — see Authentication & Multi-Tenancy) | | Permissions | `spatie/laravel-permission` (roles + granular `{slug}-{action}` permissions) | | PDF generation | `barryvdh/laravel-dompdf` | | Frontend | Next.js 15 (App Router, JavaScript only, no TypeScript) | | UI | Tailwind CSS + shadcn/ui-style components (Radix primitives), `tailwindcss-animate` | | Data fetching | `@tanstack/react-query` | | Client state | `zustand` | | Date handling | `date-fns`, `react-day-picker` | | HTTP client | `axios` |

Documentation Index

1. Architecture Overview — how the two Next.js "apps" (tenant admin panel + platform SaaS admin) and the one Laravel backend fit together.
2. Backend Structure — models, controllers, services, API route map.

3. Authentication & Multi-Tenancy — the two auth guards, and exactly how tenant data isolation is enforced.
4. Roles, Permissions & Menus — how the sidebar/top-nav is data-driven, and how access control ties into it.
5. Business Modules — CRM, Sales, Products & Services, Expenses, Reports, one section each.
6. Frontend Structure — Next.js route layout, shared component conventions, page patterns.
7. Database Schema Summary — key tables and how they relate.

Where things live

```
enscone/
├── backend/     Laravel API (routes/api.php is the single entry point)
├── frontend/    Next.js admin panel + platform SaaS admin (one app, two route groups)
└── app-doc/     ← you are here
```

Architecture Overview

Two identities, one codebase

MyBiz2Flow actually serves two distinct audiences from one Laravel backend and one Next.js frontend:

1. **Tenant Admin Panel** — what a company (a salon, a trading business, a services firm, etc.) uses day to day: CRM, quotations, invoices, expenses, reports. Routes live under `frontend/src/app/(dashboard)/...`.
2. **Platform (SaaS) Admin** — MyBiz2Flow's own operators, who manage every tenant company, subscription plans, and platform-wide settings. Routes live under `frontend/src/app/platform/...`.

These two are kept **completely separate at the auth layer** (different Sanctum guards, different login endpoints, different user tables) — see Authentication & Multi-Tenancy. A tenant user's token is rejected outright if used against a platform endpoint, and vice versa.

Request flow (tenant side)

```
Browser (Next.js, "use client" pages)
   │  axios (Bearer token from Zustand-persisted auth store)
   ▼
routes/api.php  ── auth:sanctum + CheckSubscriptionStatus middleware
   │
   ▼
Controller (Api\*Controller)  ── validates request, calls a Service where logic is non-trivial
   │
   ▼
Eloquent Model  ── BelongsToCompany trait auto-applies CompanyScope (tenant isolation)
   │
   ▼
MySQL
```

Most CRUD endpoints are plain Laravel `apiResource` routes hitting a controller directly. Anything with real business logic (document totals, number generation, PDF rendering, invoice balance recalculation, reporting aggregations) is pulled into an `app/Services/*` or

`app/Support/*` class rather than living in the controller — e.g. `ReportService`, `NumberGenerator`, `DocumentTotals`, `InvoiceBalance`, `PdfDocumentData`.

Frontend composition

The Next.js app is JavaScript-only (no TypeScript) and uses the App Router with route groups:

- `(auth)` — login/register, no sidebar chrome.
- `(dashboard)` — the tenant admin panel; `layout.jsx` here fetches the permission-filtered navigation tree once and renders the shared `Header` (top module tabs) + `Sidebar` (child-page tiles) around every page.
- `platform/` — the SaaS admin, with its own layout and its own auth store, entirely independent of the tenant `(dashboard)` layout.

Data fetching on every page is `@tanstack/react-query` calling thin `src/services/*.js` wrappers around a shared `axios` instance (`src/lib/axios.js`), which attaches the bearer token and base URL. Client-only state (the logged-in user, permission list, sidebar open/closed) lives in `zustand` stores (`src/store/*`).

Why a service layer at all

Controllers stay thin and are the easiest place to see *what* an endpoint does; anything reused across endpoints, or complex enough to unit test in isolation (VAT math, aging buckets, PDF payload shaping), is factored into a service/support class so it isn't duplicated between, say, the invoice PDF and the sales report.

Backend Structure

Everything is a single Laravel 13 app under `backend/`. There is one routes file, `routes/api.php`, split into three blocks:

1. **Public** — `POST /auth/login`, `POST /auth/register` (tenant self-signup, creates a `Company` + first `owner` user).
2. **Protected (tenant)** — everything behind `auth:sanctum` + `CheckSubscriptionStatus`. This is almost the entire file.
3. **Platform** — under `/platform/*`, behind the separate `platform` guard (see Authentication & Multi-Tenancy). Never nested inside the tenant block.

Controllers

`app/Http/Controllers/Api/` — one controller per resource, mostly thin wrappers around `apiResource` routes with Form Request validation:

`CustomerController`, `CustomerContactController`, `LeadController`, `LeadSourceController`, `LeadStageController`, `ActivityController`, `ItemController`, `UnitController`, `QuotationController`, `InvoiceController`, `PaymentController`, `PaymentMethodController`, `ExpenseController`, `ExpenseCategoryController`, `AttachmentController`, `TermTemplateController`, `TeamMemberController`, `CompanyController`, `CompanySettingController`, `SubscriptionController`, `ReportController`.

Sub-namespaces:

- `Api/Administration/` — `UserController`, `RoleController`, `PermissionController`, `MenuController`, `AuditLogController`, `StoreSettingController` (tenant-admin-only management screens).
- `Api/Auth/` — `AuthController` (tenant login/register/logout/me/profile/password).
- `Api/Dashboard/` — `DashboardController` (the CRM/sales/expense summary cards + charts on `/dashboard`).
- `Api/Platform/` — the entire SaaS-admin surface: `AuthController`, `DashboardController`, `CompanyController` (manage tenants — suspend/activate, view their users/activity), `PlanController`, `SubscriptionController`, `SystemAdminController` (platform admin accounts), `ImpersonationController` (platform admin can "log in as" a tenant user for

support), `ActivityController`, `AuditLogController`.

Services & Support classes

Business logic that's non-trivial or shared lives outside the controllers:

Class	Responsibility
`AuthService` / `PlatformAuthService`	Login/token issuance for each guard
`AuthUser`	Guard-aware "who is the current user" resolver, used by `Tenant` (see auth doc)
`DashboardService` / `PlatformDashboardService`	Aggregates behind the two dashboards
`ReportService`	Sales, expense, aging, VAT, and customer-statement report queries
`SubscriptionService`	Plan changes, cancellation, usage limits
`AuditService`	Writes `AuditLog` rows (who did what, when, from where)
`ImpersonationService`	Platform admin → tenant user impersonation sessions
`App\Support\Tenant`	Resolves "current company" for scoping — see auth doc
`App\Support\NumberGenerator`	Sequential document numbers (`QUO-2026-00001`, `INV-2026-00001`, etc.)
`App\Support\DocumentTotals`	Shared subtotal/discount/tax/total math for quotations & invoices
`App\Support\InvoiceBalance`	Recomputes an invoice's paid/balance amounts from its payment allocations
`App\Support\PdfDocumentData`	Shapes a quotation/invoice into the data a Blade PDF template expects
`App\Support\Html`	Small HTML-escaping/formatting helpers used in PDF templates

Key models

`app/Models/` (plus `Administration/` for admin-only ones):

- **Tenant/company**: `Company`, `CompanySetting`, `Subscription`, `Plan`.
- **CRM**: `Lead`, `LeadSource`, `LeadStage`, `Customer`, `CustomerContact`, `Activity`.
- **Sales**: `Quotation`, `QuotationItem`, `Invoice`, `InvoiceItem`, `Payment`, `PaymentAllocation`, `PaymentMethod`, `TermTemplate`.
- **Products**: `Item`, `Unit`.
- **Expenses**: `Expense`, `ExpenseCategory`.
- **Shared**: `Attachment` (polymorphic file uploads across customers/leads/quotations/invoices/expenses), `Administration\AuditLog`.
- **Admin/platform**: `Administration\User` (tenant users), `Administration\Menu` (drives the nav — see menus doc), `Administration\StoreSetting` (platform branding), `SystemAdmin` (platform admin accounts), `ImpersonationSession`.

Almost every tenant-owned model uses the `BelongsToCompany` trait (see next doc) — that's the single mechanism enforcing that Company A never sees Company B's rows.

Authentication & Multi-Tenancy

Two completely separate identities

MyBiz2Flow has two kinds of logged-in user, and they never overlap:

| | Tenant user | Platform admin | |---|---|---| | Model | `App\Models\Administration\User` | `App\Models\SystemAdmin` | | Guard | `sanctum` (provider: `users`) | `platform` (provider: `system_admins`) | | Login endpoint | `POST /auth/login` | `POST /platform/login` | | Sees | Only their own company's data | Every tenant company |

Both guards use Sanctum's token driver, but each token is bound to a specific provider model. A tenant user's bearer token is **rejected outright** by any `platform.*`-protected route, and a platform admin's token is rejected by the tenant routes — there's no code path where a token issued for one identity can be replayed against the other.

Tenant data isolation

This is the part that makes "log in as Company 2, only ever see Company 2's rows" work automatically, everywhere, without each controller having to remember to filter:

1. Every tenant-owned model carries a `company_id`

`User`, `Customer`, `Lead`, `Quotation`, `Invoice`, `Expense`, `Item` — all of them.

2. `BelongsToCompany` trait (`app/Models/Concerns/BelongsToCompany.php`)

Applied to every tenant-owned model. It does two things via Eloquent's `booted()` hooks:

- **Registers a global scope** (`CompanyScope`) that runs on *every* query against that model — `all()`, `find()`, `where()`, everything.
- **Stamps `company_id` on create**, from the current tenant context — and ignores any `company_id` the request body tried to set.

3. `CompanyScope` (`app/Models/Scopes/CompanyScope.php`)

The actual filter. On every query it applies `WHERE {table}.company_id = {current tenant id}` — unless the caller is a platform super-admin (unrestricted) or there's no tenant context at

all (trusted internal code: artisan commands, queue workers, seeders — also unrestricted, since there's nothing to scope by).

If somehow authenticated with no company at all, it fails closed (`WHERE 1 = 0`) rather than risk leaking rows.

4. `Tenant` (`app/Support/Tenant.php`)

The single source of truth for "what company am I scoped to right now." `Tenant::id()` reads `company_id` straight off the authenticated user (`AuthUser::user()?->company_id`) — **never** from anything the frontend sends. A `set()` / `forget()` override exists only for trusted internal contexts (console commands acting on behalf of a specific company).

Net effect

Because the scope lives on the model rather than in each controller:

- A new endpoint can't forget to filter by tenant — it's already impossible to bypass without deliberately dropping to raw SQL.
- Tampering with an `{id}` in a URL (`GET /customers/47`) can't leak another tenant's row — the scoped query simply won't find it, so it 404s instead of returning data.
- Creating a record can't be tricked into landing in the wrong company by passing a different `company_id` in the payload — it's overwritten server-side.

Subscription gating

`CheckSubscriptionStatus` middleware sits on every tenant route (alongside `auth:sanctum`) and blocks access once a company's subscription is expired/cancelled — except for the subscription/plan endpoints themselves, so a blocked tenant can still see their usage and pick a new plan.

Roles vs. permissions

Tenant authorization is two layers:

1. **Roles** (`spatie/laravel-permission`) — `owner` , `admin` , `sales` , `accountant` , `staff` (plus the platform-only `super-admin`). Some routes are gated directly by role (e.g. only `admin|owner` can manage other users; only `owner|admin|accountant|demo` can view

reports).

2. **Granular permissions** — `{slug}-{action}` pairs (e.g. `crm-leads-view`, `sales-invoices-edit`) generated automatically for every navigable menu item. CRM/Sales/Products/Expenses endpoints are gated by these via Policies, not by role directly, so a role's actual access always matches what's configured in `MenuSeeder`'s `$rolePermissions` map — see Roles, Permissions & Menus.

Roles, Permissions & Menus

The sidebar/top-nav isn't hardcoded in the frontend — it's data, generated from `database/seeders/MenuSeeder.php`, filtered per-user by permission, and rendered generically by `Header.jsx` / `Sidebar.jsx`.

The menu tree is the single source of truth

`MenuSeeder.php` defines a `$modules` array — a tree of module definitions:

```
[
    'key' => 'crm',
    'name' => 'CRM',
    'icon' => 'UsersRound',
    'sort_order' => 2,
    'children' => [
        ['key' => 'leads', 'name' => 'Leads', 'route_name' => '/leads', 'icon' => 'Target', 'sort_o
        ['key' => 'customers', 'name' => 'Customers', 'route_name' => '/customers', 'icon' => 'Cont
        ...
    ],
],
```

Rules:

- A node **with `children` and no `route_name`** is a *parent module* — it becomes one of the colored tabs in the top `Header` (CRM, Sales, Products & Services, Expenses, Reports, Settings), and its children become the tile grid in the `Sidebar` once that tab is active.
- A node **with a `route_name` and no children** is a stand-alone page (e.g. `Dashboard`) — rendered as a tile directly, not behind a parent tab.
- The seeder recursively `slug`s each node by joining ancestor keys, e.g. `admin` → `admin-users`. This slug is what permissions are generated against.

On `db:seed --class=MenuSeeder`, the seeder truncates and rebuilds `menus`, `permissions`, `role_has_permissions`, and `model_has_permissions` from this array — it's meant to be edited and re-run as the canonical way to change navigation structure, not hand-edited via SQL. (There is also a live "Menus" CRUD screen at `/menus` for ad-hoc changes without a redeploy, but the seeder is what ships with the codebase.)

Permission generation

For every menu node that has a `route_name`, four permissions are auto-created: `{slug}-view`, `{slug}-create`, `{slug}-edit`, `{slug}-delete`. So `Leads` (`slug = crm-leads`) gets `crm-leads-view`, `crm-leads-create`, `crm-leads-edit`, `crm-leads-delete`.

Role → permission mapping

Also in `MenuSeeder.php`, a `$rolePermissions` array explicitly lists which `{slug}-{action}` pairs each role gets:

```
'sales' => [
    'crm-leads'        => ['view', 'create', 'edit', 'delete'],
    'crm-customers'    => ['view', 'create', 'edit'],
    'sales-invoices'   => ['view', 'create'],
    ...
],
```

This is the **single file to check or edit** when you need to know or change "what can role X do." Roles: `owner`, `admin` (both effectively run the company), `sales`, `accountant`, `staff`. `super-admin` is platform-only and is synced *every* permission that exists (bypasses this map entirely).

How the frontend nav is built

1. On load, `(dashboard)/layout.jsx` calls `useNavigationMenu()`, which hits `GET /menus/navigation`.
2. `MenuController::navigation()` → `MenuRepo::getNavigation($user)`:
    - Loads every active menu.
    - Section headers (no `route_name`) are always kept.
    - Navigable items require `{slug}-view` — kept only if the permission doesn't exist, or the user has it.
    - Builds the tree, then drops any parent section left with zero children (so an empty module tab never renders).

3. `Header.jsx` renders one tab per item that has `children` and no `route_name`. `Sidebar.jsx` renders the *active* module's children as a tile grid (or a collapsible nested list for a child that itself has children, like Settings' sub-items).
4. `layout.jsx` also flattens the returned tree into a route whitelist (`flattenRoutes`) — if the current URL isn't in it, the page renders an `UnauthorizedView` instead of the page content. This is a UX safety net; the *real* enforcement is still server-side (the permission check on the API call itself).

Practical implication

Adding a genuinely new module (not just a new field on an existing page) means:

1. Add it to `$modules` in `MenuSeeder.php` (with `children` if it should be a top-nav tab).
2. Add its slug to whichever roles should see it in `$rolePermissions`.
3. Re-run `php artisan db:seed --class=MenuSeeder`.
4. Build the actual Next.js page(s) at the `route_name` path(s) — the nav will only ever link to routes that already have a `page.jsx`, the seeder doesn't create pages.

This is exactly the pattern used to add the Reports module as a proper parent tab with five child report pages (see `app-doc/05-business-modules.md#reports`).

Business Modules

CRM

Leads (`/leads`) — a prospect, with a `source` (Google, Referral, Walk-in, …, managed under Lead Sources), a `stage` (a company-configurable pipeline stage, e.g. New → Contacted → Quotation Sent → Won/Lost), an `assignee`, an `estimated_value`, and a `status` of `open | won | lost`. A lead can be **converted to a customer** (`POST /leads/{lead}/convert-to-customer`), which creates a `Customer` row and marks the lead `won`.

Customers (`/customers`) — individual or company accounts. Each can have multiple `CustomerContact` rows (for company-type customers with several contact people), a credit limit, payment terms, and rolls up its own quotations/invoices/payments/activities on its detail page.

Activities — a polymorphic log (call, meeting, email, WhatsApp, note, follow-up) attachable to a Lead or Customer, shown as a tabbed "Log Activity / Timeline" widget (`ActivityFeed` component) on both detail pages.

Sales

The core document flow is **Quotation → Invoice → Payment**:

1. **Quotations** (`/quotations`) — line items priced from `Items` (or free-text), with per-line and document-level discount/tax. Status lifecycle: `draft → sent → viewed → accepted | rejected`, or `expired` if `valid_until` passes unaccepted. An accepted quotation can be **converted to an invoice** (`POST /quotations/{quotation}/convert-to-invoice`), copying all line items over.
2. **Invoices** (`/invoices`) — same line-item structure as quotations, plus a `due_date`. Status: `draft → sent → partially_paid | paid`, or `overdue` (computed from `due_date` vs. today, not a stored transition), or `cancelled`.
3. **Payments** (`/payments`) — a payment is recorded against a customer and then *allocated* across one or more of their open invoices (`PaymentAllocation`). `InvoiceBalance` support class recalculates each invoice's `paid_amount` / `balance_amount` /status whenever an allocation changes.

Both quotations and invoices can be downloaded/viewed as PDF (`GET /quotations/{id}/pdf` , `GET /invoices/{id}/pdf}`) via `PdfDocumentData` + `barryvdh/laravel-dompdf` . Document numbers (`QUO-2026-00001` , `INV-2026-00001`) are sequential per company per year, generated by `App\Support\NumberGenerator` .

Term Templates — reusable "terms & conditions" blocks an owner/admin can define once and pick from when creating a quotation/invoice.

Products & Services

Items (`/items`) — the catalog quotations/invoices pull line items from: name, SKU, unit price, default tax rate, and a `Unit` (pieces, kg, hours, ...). Units and Payment Methods follow the same pattern: platform-wide defaults plus per-company custom entries, managed inline from the Items/Payments pages respectively rather than as their own top-level nav item.

Expenses

Expenses (`/expenses`) — company spending, categorized (`ExpenseCategory` , company-managed), with a vendor, amount, tax, and optional receipt attachment. Feeds directly into the Expense Report and the VAT Summary's input-VAT side.

Reports

Reports (`/reports/*`) is a **parent nav module** with five child pages, each hitting its own `ReportController` endpoint via `ReportService` :

| Page | Endpoint | What it shows | |---|---|---| | Sales Report | `GET /reports/sales` | Invoiced/paid/outstanding totals + invoice list, for a date range | | Expense Report | `GET /reports/expenses` | Spend by category + expense list, for a date range (optionally filtered to one category) | | Aging / Outstanding | `GET /reports/aging` | Unpaid invoice balances bucketed into Current / 1-30 / 31-60 / 61-90 / 90+ days overdue | | VAT Summary | `GET /reports/vat` | Output VAT (from sales) vs. input VAT (from expenses) vs. net payable, by month, for a date range | | Customer Statement | `GET /reports/customer-statement` | Running ledger (opening balance → invoices/payments → closing balance) for one selected customer |

Access is restricted server-side to `owner|admin|accountant|demo` roles (`routes/api.php`), independent of the menu-permission system used elsewhere, since these expose the

company's financials directly.

Each report page fetches its whole (already date-filtered) dataset in one request rather than a paginated resource, then renders its table with `ReportTable` — a client-side-only counterpart to the app's `ServerDataTable` (same look: search, sortable headers, pagination), since re-sorting/paging an already-loaded small dataset locally is both simpler and faster than round-tripping to the server for it.

Attachments (cross-cutting)

Files can be attached to Customers, Leads, Quotations, Invoices, and Expenses via one polymorphic `Attachment` model and a shared `AttachmentPanel` frontend component, rather than a separate uploads table per entity.

Frontend Structure

Next.js 15, App Router, plain JavaScript (`.jsx` / `.js` , no TypeScript). Almost every page/component is `"use client"` — this is an admin panel behind auth, not a content site needing SSR/SEO.

Route groups

```
src/app/
├── (auth)/            login, register — no dashboard chrome
├── (dashboard)/        the tenant admin panel (see below)
└── platform/           the SaaS admin — entirely separate layout + auth store
```

`(dashboard)/layout.jsx`

The shell every tenant page renders inside:

- Redirects to `/login` if not authenticated (`useAuthStore`).
- Fetches the permission-filtered nav tree once (`useNavigationMenu`) and renders `Header` (top module tabs) + `Sidebar` (active module's child tiles) around `{children}` .
- Tracks `activeModuleId` so the sidebar keeps showing the last-active module's tiles even on pages (like `/dashboard`) that don't belong to any module.
- Gates the page itself: if the current path isn't in the permission-filtered route list, renders `UnauthorizedView` instead of `children` .

Page folders

One folder per route under `(dashboard)/` : `leads` , `customers` , `activities` , `quotations` , `invoices` , `payments` , `items` , `expenses` , `reports/{sales,expenses,aging,vat,customer-statement}` , `users` , `roles` , `menus` , `audit-logs` , `settings` , `dashboard` , `profile` . A detail page is `{folder}/[id]/page.jsx` ; a dedicated create/edit page (where the form is too big for a modal) is `{folder}/new/page.jsx` or `{folder}/[id]/edit/page.jsx` .

Shared component conventions

| Component | Used for | |---|---| | `HomePageWrapper` (`components/shared/`) | Wraps every **list** page: renders the breadcrumb, then a soft-bordered card (`max-w-7xl` by default) around the table | | `PageWrapper` (`components/shared/`) | Wraps every **detail/form** page: same breadcrumb + card treatment, with an inner max-width constraint for the content | | `DynamicBreadcrumb` | Renders the `Home > Section > Page` trail; takes an `items` array, used internally by both wrappers above (pass `breadcrumb={...}` to the wrapper — don't render this directly) | | `ServerDataTable` (`components/Datatable/`) | The standard list-page table: server-side search/sort/pagination against a Laravel paginated endpoint | | `ReportTable` (`components/Datatable/`) | Visual twin of `ServerDataTable` for pages whose full dataset is already in memory (the Reports pages) — search/sort/paginate run client-side, no extra network round trip | | `Modal` (`components/Elements/`) | Wraps shadcn `Dialog` with a sticky header/footer (content scrolls between them), consistent sizing presets, and an optional `overlay={false}` to hide the backdrop | | `ConfirmDialog` (`components/shared/`) | Wraps shadcn `AlertDialog` for delete/destructive-action confirmations | | `ActivityFeed` (`components/shared/`) | The Log Activity / Timeline tabbed widget on Lead & Customer detail pages | | `AttachmentPanel` (`components/shared/`) | File upload/list widget, works against the polymorphic `Attachment` API for any entity type | | `DatePicker` (`components/forms/`) | Calendar-popover date field (`react-day-picker`-based), drop-in for a plain `YYYY-MM-DD` string — the standard date input across forms | | `FormField` / `TextInput` / `SelectInput` / `Fieldset` (`components/forms/`) | Standard label+input+error wrapper used by every form |

Data & state

- **Server data**: `@tanstack/react-query`, called through `src/services/*.js` — one file per resource, each a thin wrapper around the shared `axios` instance (`src/lib/axios.js`, which injects the bearer token and base URL from the auth store).
- **Client state**: `zustand` — `store/authStore.js` (tenant user, permissions, `can()` helper) and `store/platformAuthStore.js` (platform admin, entirely separate).
- **Currency/company formatting**: `useCompanyCurrencyFormat()` / `useCurrencyFormat()` hooks read the tenant's configured currency so amounts format consistently everywhere.
- **Permissions in components**: `usePermission()` hook exposes `can("{slug}-{action}")`, used to conditionally render Edit/Delete buttons per row — this is a UX nicety only; the API enforces the real check independently.

Styling

Tailwind CSS, with shadcn/ui-style primitives in `components/ui/` (Radix-based: Dialog, AlertDialog, Select, Popover, Dropdown-menu, Tabs, Calendar, etc.) plus `tailwindcss-animate` for the `animate-in` / `fade-in-0` / `zoom-in-95` transition utilities those primitives use. A `cn()` helper (`clsx` + `tailwind-merge`) is used everywhere className logic branches.

Database Schema Summary

Not a full column-by-column reference (the migrations in `backend/database/migrations/` are the source of truth for that) — this is a map of how the tables relate, grouped by area.

Platform / tenancy

```
plans ──< subscriptions >── companies ──< company_settings
                                │
                                ├──< users (tenant staff — Administration\User)
                                └──< store_settings  (platform-wide branding, not per-company)

system_admins            (platform operators — separate from `users` entirely)
impersonation_sessions   (system_admin → user, for support access)
```

Every table below except `plans`, `system_admins`, and Spatie's permission tables carries a `company_id` foreign key and is filtered by `CompanyScope` (see Authentication & Multi-Tenancy).

Access control

```
roles / permissions / model_has_permissions / role_has_permissions / model_has_roles   (spatie/lara
menus   (self-referencing parent_id — drives nav; see 04-roles-permissions-and-menus.md)
audit_logs   (user_id or system_admin_id + company_id, action, form, record_id, ip, browser)
```

CRM

```
lead_sources, lead_stages   (company-configurable lookup lists)
leads ── belongs to lead_sources, lead_stages, users (assignee) ── may reference a converted custom
customers ──< customer_contacts
activities   (polymorphic: attachable to leads or customers via related_type/related_id)
```

Sales

```
term_templates          (reusable T&Cs text, company-owned)
quotations ──< quotation line items (defined in the same migration as `quotations`)
     │  belongs to customers
     └── convert-to-invoice ──> invoices
invoices ──< invoice line items (defined alongside `invoices`)
     │  belongs to customers, optionally traces back to the source quotation
payment_methods          (platform defaults + per-company custom)
payments ──< payment allocations (defined alongside `payments`) >── invoices
     │  belongs to customers
```

A payment is created against a customer, then allocated across one or more of that customer's invoices; `InvoiceBalance` recomputes each invoice's paid/balance/status whenever allocations change.

Products & Expenses

```
units            (platform defaults + per-company custom)
items ── belongs to units

expense_categories   (company-owned)
expenses ── belongs to expense_categories
```

Cross-cutting

```
attachments   (polymorphic — attachable_type/attachable_id: customers, leads, quotations, invoices,
```

Infrastructure tables

`personal_access_tokens` (Sanctum), `sessions`, `cache`, `jobs` — framework plumbing, not application data.